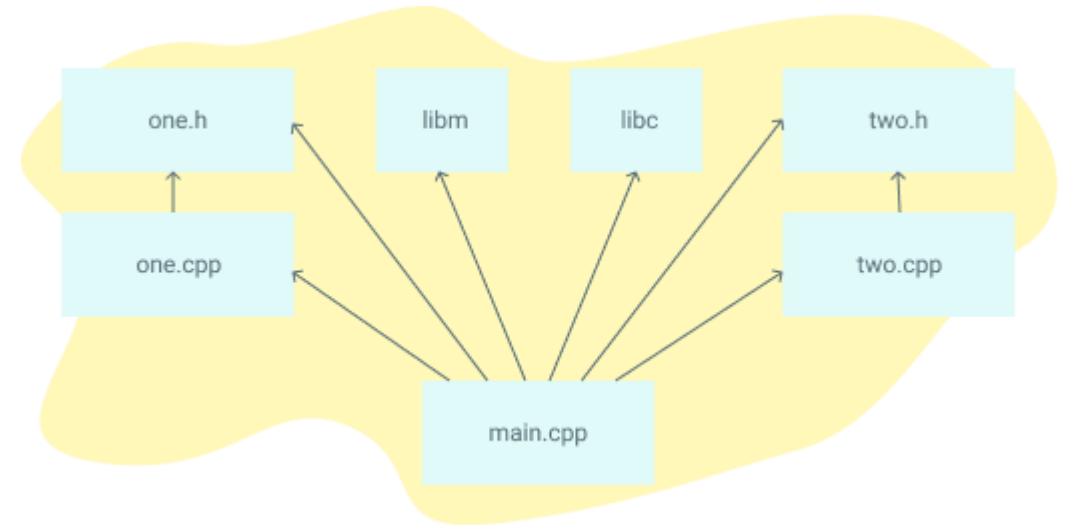


Sistem Programlama

DR. ÖĞR. ÜYESİ ABDULLAH SEVİN

makefile

- ❑ **make**, bir çok değişik çıktı üretebilen genel bir araçtır.
- ❑ Oluşturulan bir **kural** dosyası aracılığıyla, belirtilen sistem komutlarını art arda çalıştırılır.
- ❑ Bu vesile ile, çok karmaşık işleri, komut satırından verilen basit bir **make** komutu ile kolayca yapabilirsiniz.



make

- ❑ **make** için hangi alternatifler var?
- ❑ Popüler C/C++ alternatif derleme sistemleri SCons, CMake, Bazel ve Ninja'dır.
- ❑ Microsoft Visual Studio gibi bazı kod düzenleyicilerin kendi yerleşik **build** araçları vardır.
- ❑ Java için Ant, Maven ve Gradle var.
- ❑ Go ve Rust gibi diğer dillerin kendi **build** araçları vardır.

makefile

- ❑ Günümüzde açık kaynak kodlu yazılımlar, kaynak dosyalarının yanında derleme işlemi için kullanılacak **makefile** dosyası ile birlikte gelmektedir.
- ❑ Herhangi bir Linux dağıtımını ya da açık kaynak kodlu bir yazılımı **make** programı aracılığıyla kendiniz derleyebilirsiniz.
- ❑ Eğer C programcısıysanız ve **birden fazla kaynak dosyadan oluşan bir projeniz varsa** bu dosyaları elle derlemek ve bağlamak çok zaman alacaktır.
- ❑ Yaptığınız her değişiklikte gcc derleyicisi aracılığıyla, elle bir çok komut ve dosya adı girmeniz gerekmektedir.

make

- ❑ İşte make programı ve kural dosyası ile bu işlemleri otomatik olarak yalnızca konsol ekranına :

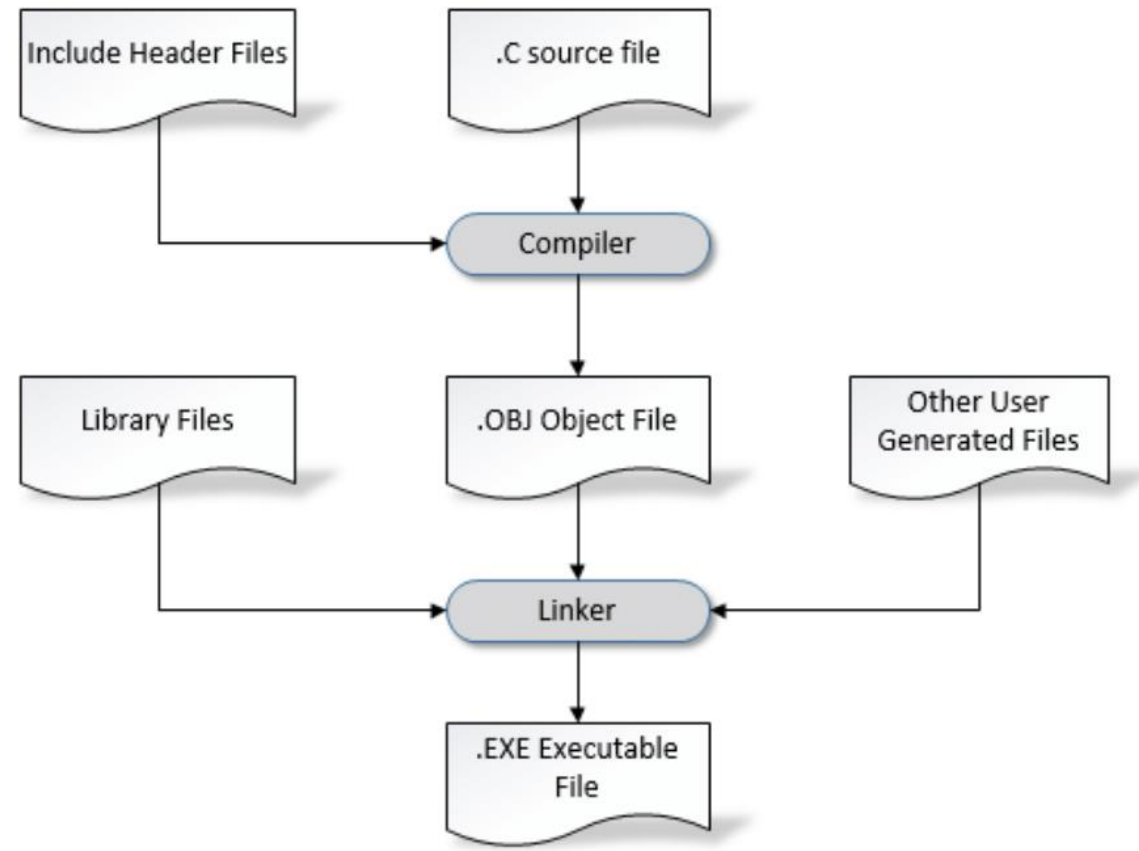
```
# make
```

komutunu yazarak yapabilirsiniz.

- ❑ Kural dosyası içerisindeki etiket ya da etiketler altındaki tanımlamalar ile **make** programı bizim için gerekli derleme ve bağlama işlemlerini yapar.
- ❑ Eğer siz kaynak dosyalar içerisinde bir **değişiklik yaptıysanız** make programı ilk önce değişiklik yaptığınız dosyayı derler, sonra da diğer obje dosyalar ile bağlayarak çalıştırılabilir dosyayı oluşturur.

Derleme-Bağlama

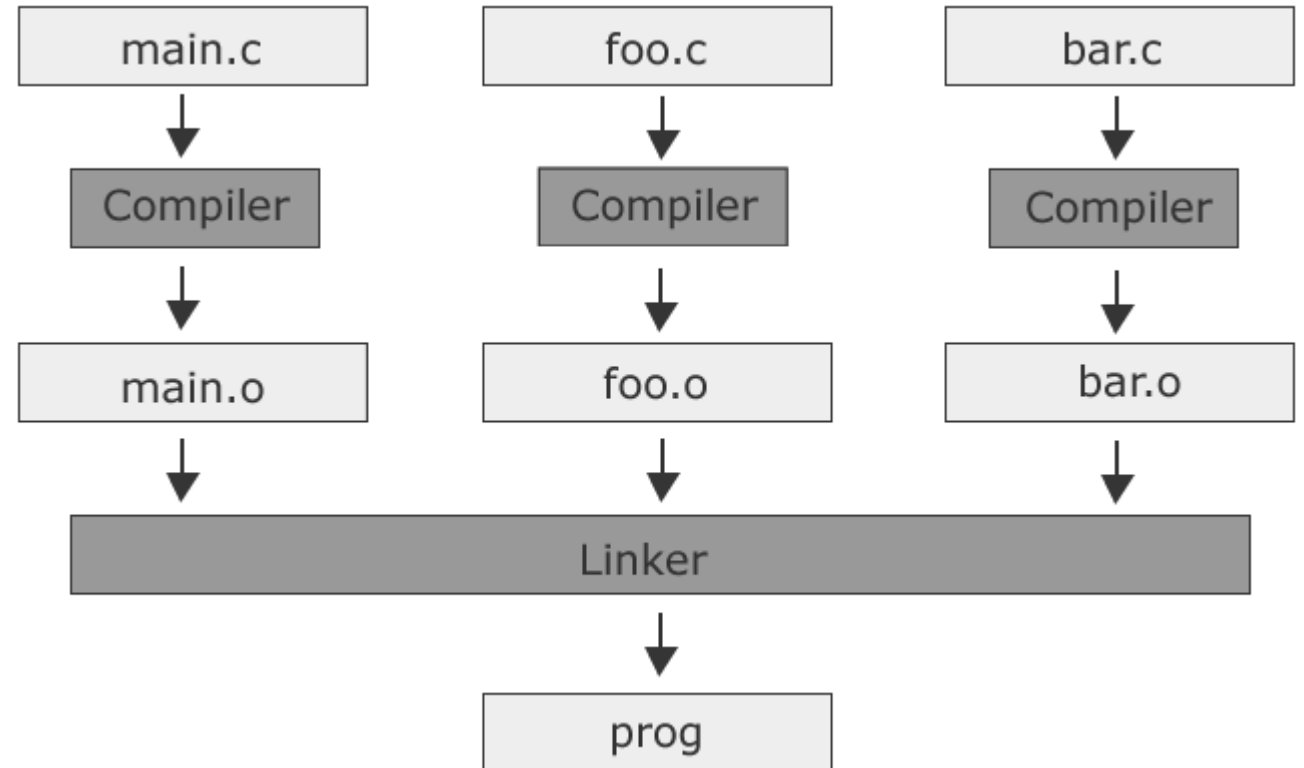
□ Basitçe bir .c dosyasından çalıştırılabilir bir dosya oluşturulması işlemi yandaki gibi olmaktadır :



https://www.researchgate.net/figure/3-Compiler-and-Linker-Relationship_fig2_320142963

makefile

□ Makefile ile birden fazla kaynak dosyasını derleme



<https://www.c-howto.de/tutorial/makefiles/>

makefile

□ Makefile dosyaları kural (rule) denilen cümleciklerden oluşturulur. Bir kuralın genel biçimi aşağıdaki gibidir :

```
Hedef : Bağımlılıklar
```

```
<tab>Komut
```

```
...
```

```
-----
```

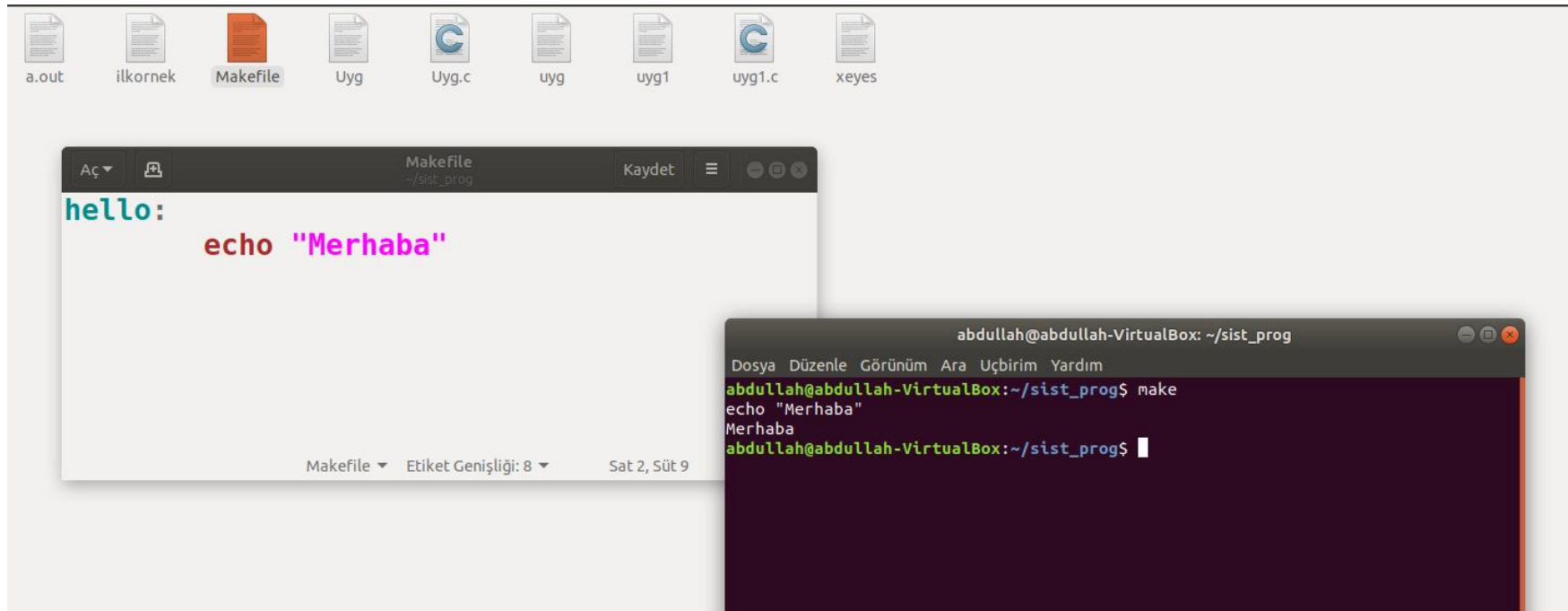
```
Hedef1 Hedef2 : Bağımlılık1 Bağımlılık2
```

```
<tab>Komut
```

```
...
```

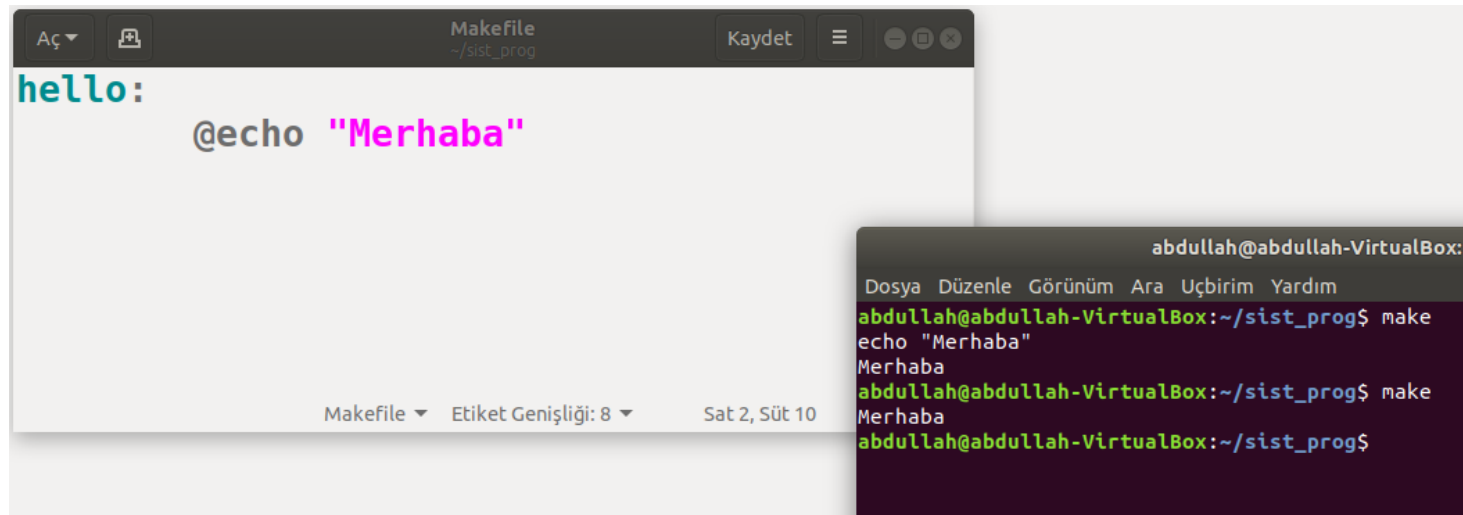

makefile

- ❑ Terminalde klasik "Merhaba Dünya" yazdırarak başlayalım.
- ❑ Bu içeriğe sahip bir Makefile dosyası içeren boş bir myproject dizini oluşturun:



make

- ❑ Yukarıdaki örneğe geri dönersek, **make** yürütüldüğünde, "Merhaba" **echo** komutunun tamamı görüntülendi ve ardından gerçek komut çıktısı geldi.
- ❑ Çoğu zaman bunu istemiyoruz. Gerçek komutun yankılanmasını engellemek için **echo** yu **@** ile başlatmamız gerekir:



The image shows a code editor window titled 'Makefile' with the content:

```
hello:
    @echo "Merhaba"
```

Below the code editor is a terminal window titled 'abdullah@abdullah-VirtualBox: ~'. It shows the following commands and output:

```
abdullah@abdullah-VirtualBox:~/sist_prog$ make
echo "Merhaba"
Merhaba
abdullah@abdullah-VirtualBox:~/sist_prog$ make
Merhaba
abdullah@abdullah-VirtualBox:~/sist_prog$
```

makefile

- ❑ Birden fazla hedef yazarsak ismini yazmamız gerekli

```
Makefile
~/sist_prog

hello:
    @echo "Hello World"

generate:
    @echo "Creating empty text files..."
    touch file-{1..10}.txt

clean:
    @echo "Cleaning up..."
    rm *.txt
```

```
abdullah@abdullah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abdullah@abdullah-VirtualBox:~/sist_prog$ make
Hello World
abdullah@abdullah-VirtualBox:~/sist_prog$ make clean
Cleaning up...
rm *.txt
rm: '*.txt' silinemedi: Böyle bir dosya ya da dizin yok
Makefile:9: recipe for target 'clean' failed
make: *** [clean] Error 1
abdullah@abdullah-VirtualBox:~/sist_prog$ make clean
Cleaning up...
rm *.txt
abdullah@abdullah-VirtualBox:~/sist_prog$
```

<https://opensource.com/article/18/8/what-how-makefile>

makefile

❑ Birden fazla kaynak dosyasını **make** etmek;

```
proje : main.o io.o data.o /usr/lib/libc.a
```

```
    gcc -o proje main.o io.o data.o /usr/lib/libc.a/libc.a
```

```
main.o : data.h io.h main.c
```

```
    gcc -c main.c
```

```
io.o : io.h io.c
```

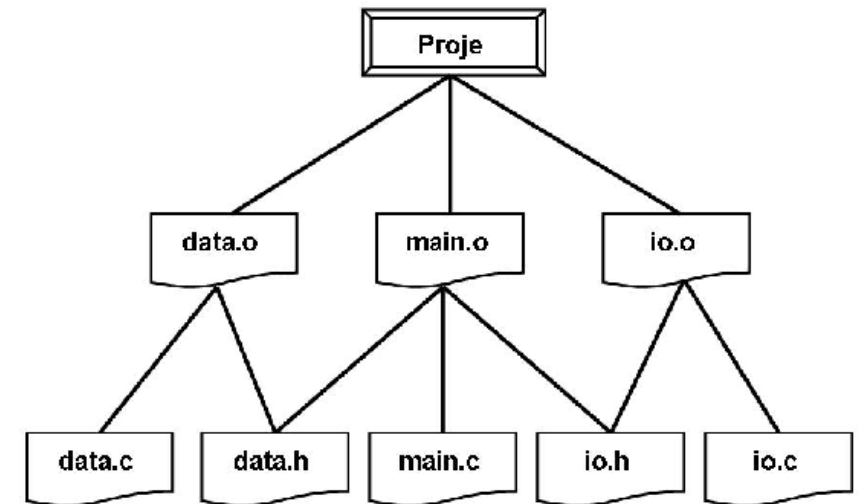
```
    gcc -c io.c
```

```
data.o : data.h data.c
```

```
    gcc -c data.c
```

```
clean :
```

```
    /usr/lib/rm *.o -c io.c
```



make

❑ **make** programı # ile başlayan satırları dikkate almaz. Buraya açıklama yazabiliriz

❑ Uyarı mesajı için;

```
proje : main.o io.o data.o /usr/lib/libc.a
```

```
cc -o proje main.o io.o data.o /usr/lib/libc.a
```

```
@echo == Derleme islemi basari ile tamamlandi!.. ==
```

Make makro

- ❑ **Makefile** dosyalarında bazı uzun yazımları engellemek için makrolar kullanılmaktadır. Makro bir çeşit C' deki sembolik sabit gibidir. Makroların temel söz dizimi şöyledir :

isim : değer\ler Örn kullanım: \${MAKRO_ISMI}

- ❑ makro kullansaydık dosyamız ;

```
LIBCDIR = /usr/lib/
```

```
COMPILER = cc
```

```
OBJS = main.o io.o data.o
```

```
EXE = proje
```

```
${EXE} : ${OBJS} ${LIBCDIR}libc.a
```

```
${COMPILER} -o ${EXE} ${OBJS} ${LIBCDIR}libc.a
```

```
@echo == Derleme islemi basari ile tamamlandi!.. ==
```